

Scenario analysis on the WATER benchmark

Simon Frost

Table of contents

1 Overview	1
2 Setup	1
3 Every marginal from one calibration	2
4 Scenarios: observing versus intervening upstream	3
5 Summary	5
6 References	5

1 Overview

WATER (Jensen et al. 1989) models the control of a waste-water treatment plant: eight water-quality quantities (biological oxygen demand, Kjeldahl nitrogen and nitrate in the different tanks) at four time points fifteen minutes apart, 32 nodes, 66 arcs and about ten thousand parameters. It is far too large for the brute-force joint distribution, which makes it the right model for `BayesianNetworkInference.jl`. This vignette computes every marginal by junction-tree calibration, cross-checks one against variable elimination, and runs scenarios that set an upstream quantity either by observation or by intervention. Scenario analysis of this kind – an upstream quantity set to a value and the consequences read off downstream – is the standard use of Bayesian networks in environmental risk assessment (Moe et al. 2021; Kaikkonen et al. 2021).

2 Setup

```
using EcologicalBayesianNetworks
using BayesianNetworks
using BayesianNetworkInference
water = load_model("water")
model_summary("water")
```

```
ModelSummary water (WATER: waste-water treatment plant control)
  format / licence      net / CC BY-SA 3.0 (verbatim)
  nodes                 32 (32 chance, 0 decision, 0 utility)
  arcs                  66
  largest state space    4
  largest in-degree      5
  load_model             BayesModel
```

The file is the gzipped HUGIN .net from the bnlearn repository ([Scutari 2010](#)), decompressed in memory; some rows sum to one only within 1e-7, so the manifest sets `renormalize = true` and the model validates.

```
validate(water; closed = true, unique_names = true, semantics = true)
```

The eight quantities at 12:00 are the roots; each feeds its own value and one or two neighbours at 12:15, and so on.

```
bn = syntax(water)
[variable_name(bn, r) => variable_name.(Ref(bn), children(bn, r)) for r in
roots(bn)]
```

```
8-element Vector{Pair{Symbol, Vector{Symbol}}}:
 :C_NI_12_00 => [:C_NI_12_15, :CBODD_12_15]
 :CKNI_12_00 => [:CKNI_12_15, :CBODD_12_15, :CKND_12_15]
 :CBODD_12_00 => [:CBODD_12_15, :CNOD_12_15, :CBODN_12_15]
 :CKND_12_00 => [:CKND_12_15, :CKNN_12_15]
 :CNOD_12_00 => [:CBODD_12_15, :CNOD_12_15, :CNON_12_15]
 :CBODN_12_00 => [:CBODD_12_15, :CBODN_12_15, :CNON_12_15]
 :CKNN_12_00 => [:CKND_12_15, :CKNN_12_15, :CNON_12_15]
 :CNON_12_00 => [:CNOD_12_15, :CBODN_12_15, :CNON_12_15]
```

3 Every marginal from one calibration

`all_marginals` with the `JunctionTree` backend builds a clique tree of the moralised graph, calibrates it once by Shafer-Shenoy message passing ([Shafer and Shenoy 1990](#)) and reads every single-variable posterior off its clique. The first call pays Julia's compilation; the timing below is of a second call. Every timing in this vignette is an order of magnitude, not a benchmark: it is whatever the machine that last rendered these pages managed, and it will differ on yours.

```
all_marginals(water; backend = JunctionTree())
t_jt = @elapsed ms = all_marginals(water; backend = JunctionTree())
length(ms), round(t_jt; digits = 3)
```

```
(32, 0.111)
```

```
ms[:CKNI_12_45]
```

```
Factor{Float64} over (:CKNI_12_45,) with size (3,)
```

Variable elimination answers one query at a time and is the oracle the junction tree is tested against; its diagnostics report the elimination order's largest factor and the treewidth of the moral graph.

```
p, diag = infer(water, :CKNI_12_45; backend = VariableElimination())
p.table ≈ ms[:CKNI_12_45].table, diag.treewidth, diag.max_factor_size
```

```
(true, 10, 1769472)
```

```
t_ve = @elapsed for x in variable_names(bn)
    infer(water, x; backend = VariableElimination())
end
round(t_ve; digits = 3)
```

```
1.426
```

4 Scenarios: observing versus intervening upstream

A scenario sets the Kjeldahl nitrogen entering the plant at 12:15, CKNI_12_15, to each of its three levels and asks what the same quantity and the biological oxygen demand look like at 12:45. Setting it by *observation* conditions the whole network, including the 12:00 quantities upstream of it; setting it by *intervention* cuts the arc from 12:00 and leaves the upstream quantities at their priors, so the two scenarios differ wherever a 12:00 quantity has another path to 12:45.

```
scenario = :CKNI_12_15
targets = [:CKNI_12_45, :CBODD_12_45]
println(lpad("scenario", 12), " ", lpad("mode", 9), " ",
join(lpad.(string.(targets), 30), " "))
for s in states(bn, scenario)
    obs = all_marginals(water; evidence = Dict(scenario ⇒ s), backend =
JunctionTree())
    int = all_marginals(do_intervention(water, scenario ⇒ s); backend =
JunctionTree())
    for (mode, res) in (("observe", obs), ("intervene", int))
        row = [string(round.(res[t].table; digits = 3)) for t in targets]
        println(lpad(string(s), 12), " ", lpad(mode, 9), " ", join(lpad.(row, 30),
" "))
    end
end
```

```
end
end
```

scenario	mode	CKNI_12_45	
CBODD_12_45			
20_MG_L	observe	[0.328, 0.538, 0.134]	[0.064, 0.881, 0.055, 0.001]
20_MG_L	intervene	[0.328, 0.538, 0.134]	[0.053, 0.863, 0.083, 0.001]
30_MG_L	observe	[0.224, 0.552, 0.224]	[0.023, 0.831, 0.141, 0.006]
30_MG_L	intervene	[0.224, 0.552, 0.224]	[0.023, 0.83, 0.141, 0.006]
40_MG_L	observe	[0.134, 0.538, 0.328]	[0.006, 0.741, 0.234, 0.019]
40_MG_L	intervene	[0.134, 0.538, 0.328]	[0.013, 0.761, 0.211, 0.015]

Upstream of the scenario variable the difference is stark: observation updates the 12:00 nitrogen, intervention leaves it alone.

```
(prior = round.(ms[:CKNI_12_00].table; digits = 3),
 observe = round.(all_marginals(water; evidence = Dict{scenario =>
Symbol("40_MG_L")}):CKNI_12_00].table; digits = 3),
 intervene = round.(all_marginals(do_intervention(water, scenario =>
Symbol("40_MG_L"))):CKNI_12_00].table; digits = 3))
```

```
(prior = [0.333, 0.333, 0.333], observe = [0.056, 0.278, 0.667], intervene = [0.333, 0.333, 0.333])
```

Each scenario is one junction-tree calibration, which on this network is a fraction of a second, so a sweep over every state of every 12:15 quantity - 29 interventional models in all - runs in seconds rather than minutes. The point is the order of magnitude: a 32-variable network with treewidth in the single digits is small enough that scenario analysis needs no special machinery.

```
slice = [x for x in variable_names(bn) if endswith(string(x), "_12_15")]
n = sum(length(states(bn, x)) for x in slice)
t = @elapsed for x in slice, s in states(bn, x)
    all_marginals(do_intervention(water, x => s); backend = JunctionTree())
end
n, round(t; digits = 3)
```

```
(29, 3.096)
```

The intervened model is an ordinary model whose history records what was rewritten, so a scenario can be drawn, saved or composed like any other:

```
history(do_intervention(water, scenario => Symbol("40_MG_L")))
```

```
1-element Vector{ModelEvent}:
ModelEvent(:hard, :CKNI_12_15, MechanismRecord(:CKNI_12_15_mechanism,
NamedRef("CKNI_12_15_mechanism"), [:CKNI_12_00]),
MechanismRecord(Symbol("do[CKNI_12_15=40_MG_L]"), PointMassRef(Symbol("40_MG_L")),
Symbol[]), "", Dates.DateTime("2026-09-07T15:20:04.276"))
```

5 Summary

WATER is large enough that the brute-force joint is out of reach and small enough that one junction-tree calibration produces every marginal in a fraction of a second, which is the case the exact backends of `BayesianNetworkInference.jl` are built for. Setting an upstream quantity by observation and by intervention gives different answers upstream and the same answers downstream, and the intervened model records what was rewritten in its history, so a scenario is an ordinary model that can be saved or composed. The last vignette, *A roadmap for dynamic models*, turns to networks with feedback across time.

6 References

- Kaikkonen, Laura, Tuuli Parviainen, Mika Rahikainen, Laura Uusitalo, and Annukka Lehikoinen. 2021. "Bayesian Networks in Environmental Risk Assessment: A Review." *Integrated Environmental Assessment and Management* 17 (1): 62–78. <https://doi.org/10.1002/ieam.4332>.
- Moe, S. Jannicke, John F. Carriger, and Miriam Glendell. 2021. "Increased Use of Bayesian Network Models Has Improved Environmental Risk Assessments." *Integrated Environmental Assessment and Management* 17 (1): 53–61. <https://doi.org/10.1002/ieam.4369>.
- Scutari, Marco. 2010. "Learning Bayesian Networks with the bnlearn R Package." *Journal of Statistical Software* 35 (3): 1–22. <https://doi.org/10.18637/jss.v035.i03>.
- Shafer, Glenn R., and Prakash P. Shenoy. 1990. "Probability Propagation." *Annals of Mathematics and Artificial Intelligence* 2: 327–51. <https://doi.org/10.1007/BF01531015>.